When Is a Temporal Statement Store a Statement Graph?

Export Boundaries for Occurrence Records with Two Clocks

Volodymyr Pavlyshyn
Independent researcher
pavlyshyn@tiramemsu.com tiramemsu.com

Revision 3, 11 October 2026

Abstract

A temporal statement store keeps every statement occurrence as a record with an identifier, a triple, a transaction-time lifetime and a valid-time interval, and lets other records refer to it. We ask which part of such a store can be exported as a Statement Graph, the DAG model that underlies OneGraph interoperability between RDF-star and property graphs, without inventing or renaming any occurrence. The answer has three parts. *What can be exported:* a selected record set admits an identity-preserving export exactly when it is closed under references and acyclic, and every set contains a greatest such subset, the least fixpoint of a one-line operator, computed by a linear-time topological peel that also yields an export order. When content is immutable, records are never deleted and inserts reference only existing records, the whole history is closed, so exporting the history rather than a snapshot loses only cycles created by atomic correction, and a per-record bit maintained at insert time turns export into a filter scan. *What is dropped:* snapshot export additionally drops supersession lineage, whose targets are retracted; a dual reference-closure keeps it by adding the retracted targets. *What is reported:* every omitted record with a reason. An explicit encoding into the RDF-star branch of the OneGraph image, with transaction and valid time as reserved statements, makes the roundtrip of the exportable history lossless. Proofs are by hand; a validator checks the constructions on exhaustive small stores and random protocol histories, and both counterexamples are traced through the tiramemsu engine.

1 Introduction

RDF identifies resources globally and represents a graph as a set of triples. Labeled property graphs (LPGs) give nodes and edges identity and attach properties to them. Statement-level information complicates translation between the two: occurrence identity, annotations and the meaning of updates need not survive a surface view. The OneGraph vision names these problems, including the ambiguity of deleting one visible triple backed by several occurrences [14]. Statement Graphs make the shared structure explicit as a directed acyclic graph whose internal vertices are statements with subject, predicate and object edges [4, 5].

A temporal store can hold a statement vertex and its three role edges in one record. The embedded database tiramemsu [20] uses

$$(e, s, p, o, a, d, v_1, v_2),$$

where e identifies an occurrence, (s, p, o) is its content, $[a, d)$ is its transaction-time lifetime and $[v_1, v_2)$ its valid-time interval. Subjects and objects may be identifiers of other records. The record shape suggests that the store *is* a Statement Graph. It is not, in general, and this paper says exactly when it is and what to do when it is not.

Running example. At transaction 1 we assert a route and annotate it with its distance:

$$e_1 = (\texttt{:FRA}, \texttt{:route}, \texttt{:LHR}), \qquad e_2 = (e_1, \texttt{:distance}, 655).$$

These two records translate to two statement vertices; the second points at the first. At transaction 2 the route's valid time is corrected. The engine retracts e_1 and its dependent e_2, replays them under fresh identifiers e_1', e_2', and records lineage $\ell = (e_1', \texttt{sys:supersedes}, e_1)$. The current snapshot $\{e_1', e_2', \ell\}$ is not a Statement Graph: ℓ points at the retracted e_1. The full history $\{e_1, e_2, e_1', e_2', \ell\}$ is. A different correction, whose patched object names the annotation e_2, replays into $e_1' = (\texttt{:FRA}, \texttt{:route}, e_2')$ and $e_2' = (e_1', \texttt{:distance}, 655)$, a two-cycle that neither the snapshot nor the history can export directly (Examples 1 and 2).

Contributions.

1. A formal notion of identity-preserving direct export and a theorem: it exists exactly for closed, acyclic record sets (Theorem 1).
2. The *export kernel*, the least fixpoint of $F_S(X) = \{r \in S : \mathsf{ref}(r) \subseteq X\}$, which is the unique greatest closed, acyclic subset; a linear-time peel computing it with an export order; and incremental maintenance by one bit per record (Section 4).
3. The dual *reference closure*, the least closed superset within the history, which keeps lineage that the kernel drops (Section 5).
4. Full-history results: under append-only content and no deletion the history is closed and its cycles lie inside single replay batches, so the history kernel drops only replay cycles and their dependents (Section 6).
5. An explicit encoding into the RDF-star branch of the OneGraph image, extended with reserved time statements to a lossless roundtrip of the exportable bitemporal history (Section 7).
6. A validator over exhaustive and random stores and protocol histories, and engine traces of both running-example corrections (Section 8).

The graph facts used are elementary: the kernel is the well-founded part of the reference relation, and its computation is Kahn's topological sort [12]. The contribution is their application to temporal statement stores: which subset to export, what an exporter must report, and how time travels with it.

Relation to earlier work. The first companion paper develops temporal integrity, the distinction between structural and lineage references, and correction contracts [17]; the second studies exact images of metagraph encodings [18]. A site article explains the row/vertex correspondence informally [19]. This paper reuses their occurrence model. Its new material is the export kernel and its fixpoint characterization, the reference closure, the full-history results and the temporal encoding.

2 Statement Graphs and their RDF-star image

We use the definitions of Gelling, Fletcher and Schmidt in the arXiv version [4]; the DASFAA paper [5] is its published form.

Definition 1 (Statement Graph [4, Def. 4.1])**.** A Statement Graph is a finite directed acyclic graph with injectively labeled vertices. An internal vertex is labeled by a statement identifier and has exactly one outgoing edge of each role: subject, predicate, object. A leaf is labeled by a concrete value, has no outgoing edges and at least one incoming edge.

The RDF-star image [4, Def. 5.1] adds typing and membership conditions. Subject targets are statements, IRIs or blank nodes; predicate targets are IRIs; object targets are statements, IRIs, blank nodes or literals. A reserved IRI *in* marks *membership statements* (v, in, g), which assign a triple statement v to a graph name or the default graph identifier g. A membership statement has a statement subject, an IRI or blank-node object and no incoming edges. Every other statement vertex has at least one incoming edge, and no two vertices are isomorphic. The membership condition is what distinguishes an asserted triple, which some membership places in a graph, from a triple that only occurs as a quoted term. The OneGraph image [4, Def. 5.4] requires each connected component to be an RDF-star image or an LPG image, disregarding the isomorphism condition. We target its RDF-star branch.

The Neptune Analytics demo uses OneGraph concepts, relationship and property statements and openCypher updates over RDF [21]; nothing below depends on its internal model.

3 Records and identity-preserving export

Let $\mathcal{T}$ be the disjoint union of IRIs $\mathcal{I}$, local node identifiers, blank nodes, literals $\mathcal{L}$ and transaction identities, and let $\mathcal{E}$ be a disjoint set of occurrence identifiers (eids). Valid time ranges over $\mathbb{Z} \cup \{-\infty, +\infty\}$; transaction numbers are positive integers.

Definition 2 (Record, store, snapshot, references)**.** A record is $r = (e, s, p, o, a, d, v_1, v_2)$ with $e \in \mathcal{E}$, $s \in (\mathcal{T} \setminus \mathcal{L}) \cup \mathcal{E}$, $p \in \mathcal{I}$, $o \in \mathcal{T} \cup \mathcal{E}$, $a \geq 1$, $d = \bot$ or $d \geq a$, and $v_1 < v_2$. A store D is a finite set of records with distinct eids. Its committed snapshot at t is $D@t = \{r \in D : a_r \leq t \text{ and } (d_r = \bot \text{ or } t < d_r)\}$. The references $\mathsf{ref}(r) \subseteq \mathcal{E}$ are the eid endpoints of r, and $r \prec r'$ when $e_{r'} \in \mathsf{ref}(r)$. A set $S \subseteq D$ is *closed* if $\mathsf{ref}(r) \subseteq e(S)$ for every $r \in S$, and *acyclic* if $\prec$ restricted to S has no directed cycle.

Write $\mathsf{erase}(S) = \{(e, s, p, o) : r \in S\}$ for temporal erasure; unlike projection to (s, p, o), it keeps occurrence identity. Fix an injective map f from source terms to target terms that preserves literals and blank nodes and sends every other term to an IRI.

Definition 3 (Identity-preserving direct export)**.** An identity-preserving direct export of S is a Statement Graph G such that (a) the internal vertices of G are exactly one vertex labeled e_r for each $r \in S$; and (b) for each $r \in S$ and role $\rho \in \{s, p, o\}$, if $r.\rho$ is an eid x, the ρ-edge of e_r targets the internal vertex labeled x, and otherwise it targets a leaf labeled $f(r.\rho)$.

Condition (a) forbids inventing occurrences, including referents absent from S. Condition (b) forbids rewriting a reference into anything other than the referenced occurrence.

Theorem 1 (Exact direct-export criterion)**.** *S has an identity-preserving direct export if and only if S is closed and acyclic. The export is then unique; call it $\mathsf{SG}(S)$. Reading the three role targets of each internal vertex and applying f^{-1} to leaves recovers $\mathsf{erase}(S)$.*

Proof. ($\Leftarrow$) Build one internal vertex per record and one leaf per distinct term $f(t)$ occurring in S, and add the role edges required by (b). Closure supplies every internal target. Distinct eids, injectivity of f and disjointness of $\mathcal{E}$ and $\mathcal{T}$ give injective labels; each leaf has an incoming edge by construction. Edges between internal vertices are exactly references, and leaves have no outgoing edges, so the graph is acyclic because S is.

($\Rightarrow$) Let G satisfy (a) and (b). If $x \in \mathsf{ref}(r)$ for some $r \in S$, (b) requires an internal vertex labeled x, and by (a) that is $e_{r'}$ for some $r' \in S$; hence S is closed. By (b) every reference $r \prec r'$ is an edge between internal vertices, so a reference cycle in S is a cycle in G, which a Statement Graph does not have.

Uniqueness: (a) and (b) fix all internal vertices and all edges; a leaf has an incoming edge, so the leaves are exactly the targets named in (b). Recovery reads each vertex's three targets independently. □

Remark 1 (Leaf substitution). For a nonclosed S one can still obtain a Statement Graph by reading a dangling eid x as a leaf, for instance an IRI naming x. That graph violates (b), and a consumer cannot tell a reference to an absent occurrence from an ordinary term. The validator checks this on an explicit case.

4 The export kernel

Fix a selected set S, typically a snapshot $D@t$ or the history D. We admit only the deletion of whole records: no endpoint substitution, no new referents and no change of identifiers. For $X \subseteq S$ define

$$F_S(X) = \{r \in S : \mathsf{ref}(r) \subseteq e(X)\}, \qquad \mathsf{K}(S) = \bigcup_{k \geq 0} F_S^k(\emptyset).$$

F_S is monotone, so the chain $F_S^k(\emptyset)$ increases and stabilizes after at most $|S|$ steps at the least fixpoint of F_S. The *rank* of $r \in \mathsf{K}(S)$ is the least k with $r \in F_S^k(\emptyset)$. Records without references have rank 1.

Theorem 2 (Greatest direct-export subset)**.** $\mathsf{K}(S)$ *is the unique greatest closed, acyclic subset of* S. *Consequently* $\mathsf{SG}(\mathsf{K}(S))$ *exists,* $\mathsf{K}(S) = S$ *exactly when* S *is closed and acyclic,* K *is idempotent, and* $S \subseteq S'$ *implies* $\mathsf{K}(S) \subseteq \mathsf{K}(S')$.

Proof. Closed: $\mathsf{K}(S) = F_S(\mathsf{K}(S))$, so every reference of a kernel record lands in the kernel. *Acyclic:* a record of rank k lies in $F_S(F_S^{k-1}(\emptyset))$, so its references have rank below k; references strictly decrease rank, which no cycle can do. *Greatest:* let $U \subseteq S$ be closed and acyclic. For $r \in U$ let $h(r)$ be the length of the longest reference path from r inside U; it is finite because U is finite and acyclic, and closure keeps every reference of r inside U with smaller h. By induction on h, $r \in F_S^{h(r)+1}(\emptyset)$: if $h(r) = 0$ then $\mathsf{ref}(r) = \emptyset$, and otherwise all references of r are in $F_S^{h(r)}(\emptyset)$. Hence $U \subseteq \mathsf{K}(S)$. Uniqueness of a greatest element is immediate, and the remaining claims follow: S itself qualifies exactly when it is closed and acyclic; the kernel is closed and acyclic, so it is its own kernel; and $F_S \subseteq F_{S'}$ pointwise when $S \subseteq S'$. □

The kernel is the well-founded part of $\prec$ on S, where a missing target counts as non-well-founded. It is the dual of reachability garbage collection [10]: a collector keeps what roots reach, the kernel keeps what reaches only grounded records. The obstruction form explains omissions.

Corollary 3 (Obstructions)**.** *Let* $B \subseteq S$ *consist of the records with a reference outside* $e(S)$ *and the records on a directed reference cycle in* S*, and let* A *be the records of* S *that reach* B *(zero-length paths included). Then* $\mathsf{K}(S) = S \setminus A$.

Proof. $S \setminus A$ is closed, since a retained record has no missing target and does not reach A; and acyclic, since cycle vertices are in B. Hence $S \setminus A \subseteq \mathsf{K}(S)$. Conversely a closed, acyclic set contains no record with a missing target, no cycle vertex (closure would pull in the whole cycle) and, by repeated closure, no record reaching either. □

Algorithm. For each record count its references; a record with a reference outside $e(S)$ is never ready. Queue the records whose count is zero. Repeatedly pop a record, append it to the output order, and decrement the count of each record that references it, queuing those that reach zero. The popped records are $\mathsf{K}(S)$ and the output order lists every record after all its targets, which is the order a streaming RDF-star or Statement Graph writer needs. With indexed eid lookup this takes $O(n+m)$ time, where $n = |S|$ and $m \leq 2n$ counts reference positions. Records never popped form A. Reporting a reason for each omission (missing target x, on a cycle, or depends on omitted y) needs one strongly connected component pass over A only. Revision 2 of this paper computed K from components first; the validator keeps that version as an independent implementation.

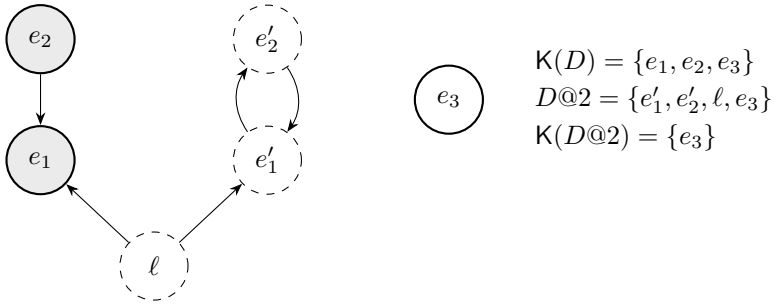


Figure 1: History of the cyclic correction (Example 2) with an unrelated fact e_3. Arrows go from referring records to their targets; term endpoints are suppressed. Shaded records are retracted at transaction 2; dashed records are outside $\mathsf{K}(D)$. The history kernel keeps the pre-correction records; the snapshot kernel keeps only e_3, because ℓ also dangles there.

What the kernel does and does not do. Closure makes removing one edge of a cycle inadmissible: both records of a two-cycle and all their dependents go, while unrelated facts stay (Figure 1). The kernel does not maximize business value or reproduce query answers over the omitted records. An exporter should return the omitted eids with reasons, and a caller that needs completeness should reject an incomplete export rather than use the kernel silently. Dropping lineage predicates by name is not a substitute: retained records may still reference dropped lineage occurrences.

5 The reference closure

The kernel deletes. The opposite design keeps every selected record and adds the historical records it references.

Definition 4 (Reference closure)**.** For $S \subseteq D$, $\mathsf{C}_D(S)$ is the set of records of D reachable from S along $\prec$, including S; it is defined when every reachable eid belongs to a record of D.

Proposition 4 (Least closed superset)**.** *If D is closed, $\mathsf{C}_D(S)$ is defined and is the least closed subset of D containing S. Moreover $\mathsf{K}(S) \subseteq \mathsf{K}(\mathsf{C}_D(S)) \subseteq \mathsf{C}_D(S)$, and every record of S in $\mathsf{K}(\mathsf{C}_D(S)) \setminus \mathsf{K}(S)$ was omitted from $\mathsf{K}(S)$ only because some record it reaches lies outside S.*

Proof. Reachability from S is closed by construction, and any closed superset of S contains everything S reaches. The inclusions are monotonicity of K (Theorem 2). For the last claim, let $r \in S \cap \mathsf{K}(\mathsf{C}_D(S))$. By Corollary 3, no record that r reaches in $\mathsf{C}_D(S)$ has a missing target or lies on a cycle. If every record r reaches were in S, then r would reach the same records in S, none of them an obstruction in S, and r would be in $\mathsf{K}(S)$. □

For $S = D@t$ the added records $\mathsf{C}_D(S) \setminus S$ are retracted at or before t, and an exporter should mark them so (Section 7 encodes this as the absence of a membership). In Example 1, $\mathsf{K}(D@2) = \{e_1', e_2'\}$ silently drops the supersession provenance ℓ, whereas $\mathsf{C}_D(D@2) = \{e_1, e_1', e_2', \ell\}$ is exportable as a whole and keeps it, with e_1 marked retracted. The reference closure therefore costs extra historical records and requires consumers to respect the retraction marks; the kernel costs provenance. A store with acyclic history can offer the closure with no omissions at all.

6 Temporal operations and the full history

A storage protocol and an export validator do different jobs: the protocol may admit structures the exporter must reject. This section states which structures a temporal store can produce. Four commitments of tiramemsu matter; the remaining design commitments are listed in Appendix A.

C3 Live references. An ordinary insert at t may reference only records live at that moment. Engine-written lineage may reference allocated historical records.

C4 Immutable history. Content and identifiers never change, records are never deleted and d is set at most once; the file enforces this with triggers.

C5 Cascade. Retracting a record retracts its live transitive dependents in the same transaction.

C6 Correction by replay. Supersession retracts the cascade set C of a root, preallocates fresh eids σ for the retained members, inserts each retained member with its references into C rewritten through σ (the root also receives the patch, itself rewritten through σ), and inserts lineage $(\sigma(\mathit{root}), \texttt{sys:supersedes}, \mathit{root})$.

6.1 Snapshots

Call the *restricted protocol* transactions numbered in increasing order whose operations are inserts obeying C3 and retractions obeying C5, with C4 throughout and no supersession.

Proposition 5 (Snapshot closure)**.** *Under the restricted protocol every committed snapshot is closed and the history is acyclic.*

Proof. Induct on operations within a transaction. An insert references live records. If a retraction removed the target of a remaining live record, that record would be a dependent and would have been retracted too. With I_t the records inserted and R_t those retracted during t, the committed snapshot is $(D@(t-1) \cup I_t) \setminus R_t$; records inserted and retracted within t have $a = d = t$ and an empty lifetime. Acyclicity: rank records by insertion; every reference points to an earlier insertion. □

Valid-time selection may still break closure: a present dispute about an expired contract can have a valid interval outside the contract's.

Example 1 (Historical lineage)**.** Correct the valid time of e_1 in the running example. Supersession retracts $\{e_1, e_2\}$, inserts e_1' with the new valid time, $e_2' = (e_1', \texttt{:distance}, 655)$ and $\ell = (e_1', \texttt{sys:supersedes}, e_1)$. Then $D@2 = \{e_1', e_2', \ell\}$ is not closed and $\mathsf{K}(D@2) = \{e_1', e_2'\}$, while the history D is closed and acyclic, so $\mathsf{K}(D) = D$.

Example 2 (Correction-generated cycle)**.** Instead supersede e_1 with an object patch naming e_2. The patch is rewritten through σ, giving $e_1' = (\texttt{:FRA}, \texttt{:route}, e_2')$ and $e_2' = (e_1', \texttt{:distance}, 655)$, plus ℓ. Neither record refers to itself; together they form a two-cycle. The engine rejects a direct self-reference but accepts this case, and its regression test pins that behavior. Here $\mathsf{K}(D) = \{e_1, e_2\}$ and $\mathsf{K}(D@2) = \emptyset$.

6.2 The full history

Exporting D rather than $D@t$ removes the dangling-lineage failure entirely.

Proposition 6 (Closed history)**.** *Under C3–C6 the history D is closed after every committed transaction, and every directed reference cycle of D consists of records inserted by one replay. Consequently $\mathsf{K}(D)$ is D minus the replay cycles and the records that reach them, and D is acyclic whenever no replay creates a cycle.*

Proof. Every reference written by an insert targets a live record (C3), a fresh eid of the same replay that is inserted in the same transaction (C6), or, for lineage, an allocated historical record. By C4 no record leaves D, so all these targets remain in D. For cycles, order the inserts of a history by position, giving all records of one replay the same position. An ordinary insert or a lineage record references strictly earlier positions; a replayed record references earlier positions or its own batch. Hence a cycle cannot leave one batch. The description of $\mathsf{K}(D)$ is Corollary 3 with no missing targets. □

Table 1: Per-operation invariants in tiramemsu 0.4.0. "Model" is checked by the protocol simulation of `validate.py`; "Engine" names engine tests.

Operation	Invariant	Checked by
Assert, create	New references $\subseteq e(D@t)$; preserves closure of $D@t$ and D; adds no cycle	Model; Engine: statement endpoints must be live
Confirm, membership	Bearer and eid graph name live; same invariant as assert	Engine documentation
Bundle import	Topological order; cyclic bundles rejected; adds no cycle	Engine: import rejects cycles
Retract	Removes all live transitive dependents; preserves closure of $D@t$; D unchanged except d	Model
Supersede	References $\subseteq \sigma(C) \cup e(D@t) \cup \{root\}$; preserves closure of D; may break closure of $D@t$; cycles only inside $\sigma(C)$	Model; Engine: correction remaps patched references; export boundary traces

Two practical consequences follow. First, a store can forbid cyclic corrections at the cost of an acyclicity check over the replay batch only, $O(|C|)$ with the peel of Section 4; then $\mathsf{K}(D) = D$ always. The present paper covers both policies; we know of no use case that needs a correction cycle, and the engine permits one because replay substitutes the patch through σ uniformly. Second, the kernel can be maintained incrementally.

Proposition 7 (Exportable bit)**.** *Assign each record a bit when its transaction commits: peel the records inserted by that transaction, treating references to earlier records as resolved exactly when their bits are set. Under C3–C6 the set bit marks exactly* $\mathsf{K}(D)$ *at every commit, and no bit ever changes. If a snapshot* $D@t$ *is closed,* $\mathsf{K}(D@t)$ *is* $D@t$ *filtered by the bit.*

Proof. A record references only records of its own or earlier transactions, so F_D restricted to earlier records is F of the earlier history, and by induction their kernel membership is unchanged by later inserts. Within the new batch the peel computes the least fixpoint with the earlier bits fixed. For a closed snapshot, everything a record of $D@t$ reaches lies in $D@t$, so by Corollary 3 its kernel membership is the same in $D@t$ and in D. □

Full-history export is thus a filter scan on one stored bit, and snapshot export is the same scan plus live filtering, except where lineage dangles. Table 1 states the per-operation invariants.

7 OneGraph encodings

We fix an encoding for any closed, acyclic S, in particular $\mathsf{K}(D@t)$, $\mathsf{K}(\mathsf{C}_D(D@t))$ and $\mathsf{K}(D)$, and then extend it with time.

Dictionary. In tiramemsu a natural choice of f keeps IRIs, literals and blank nodes and maps engine-local identifiers to IRIs scoped by the database, for instance `urn:tiramemsu:<db-uuid>:n/<id>` for nodes and `urn:tiramemsu:<db-uuid>:tx/<t>` for transaction terms. The reserved IRIs below must not occur in the image of f; source IRIs that clash are renamed and the renaming is recorded in the dictionary. The dictionary is part of the decoding contract.

Definition 5 (Structural encoding E_f and decoder R_f)**.** Emit $\mathsf{SG}(S)$ with term leaves $f(t)$. For each record e add one fresh membership statement $m_e = (e, in, G_D)$ for a reserved default-graph IRI G_D, without adding memberships to memberships. Then R_f removes the statements with predicate *in* and reads the remaining role targets, applying f^{-1} to leaves.

The memberships are required by the RDF-star image: a non-membership statement needs an incoming edge, and a record that nothing references has none. They also say that each exported record is asserted, rather than merely quoted. The cost is one extra statement vertex per record.

Theorem 8 (Image membership and structural roundtrip)**.** *$E_f(S)$ belongs to the OneGraph image, and to the RDF-star image exactly when no two records of S share (s, p, o). Moreover $R_f(E_f(S)) = \mathsf{erase}(S)$.*

Proof. Typing: subjects are nonliteral terms or statements, predicates are IRIs other than *in*, objects are any allowed term or statement. Every record vertex has an incoming membership edge; every membership has a statement subject, an IRI object and no incoming edge, since freshness prevents any record from referencing it. $\mathsf{SG}(S)$ is acyclic and memberships are new roots, so $E_f(S)$ is acyclic; labels are injective and every leaf is used. These are the RDF-star conditions other than non-isomorphism, so each component meets the OneGraph condition. Two record vertices are isomorphic exactly when their content agrees, by injectivity of f; memberships have distinct subjects and a reserved predicate, so they are never isomorphic to each other or to a record vertex. Decoding removes exactly the memberships and inverts f. □

Temporal encoding. Reserve four further IRIs $\tau_a, \tau_d, \tau_1, \tau_2$. For a closed, acyclic S and a set $\mathcal{A} \subseteq S$ of records to present as asserted, $T_f(S, \mathcal{A})$ consists of $\mathsf{SG}(S)$ with leaves $f(t)$; for each record e the time statements (e, τ_a, a), (e, τ_d, d) if $d \neq \bot$, (e, τ_1, v_1) if $v_1 \neq -\infty$ and (e, τ_2, v_2) if $v_2 \neq +\infty$, with integer literal objects and fresh identifiers; a membership for every time statement; and a membership m_e exactly for $e \in \mathcal{A}$. The decoder R^τ_f removes memberships, recording which records carried one, groups time statements by subject, and restores each record with $\bot$ or $\pm\infty$ for absent clocks.

Theorem 9 (Lossless temporal roundtrip)**.** *$T_f(S, \mathcal{A})$ belongs to the OneGraph image, and to the RDF-star image exactly when no two records of S share (s, p, o). Moreover $R^\tau_f(T_f(S, \mathcal{A})) = (S, \mathcal{A})$, with all eight fields of every record.*

Proof. The time statements have a record subject, a reserved IRI predicate and a literal object, are referenced by nothing but their own memberships, and so add neither cycles nor dangling edges. Every record vertex has an incoming edge from its τ_a statement whether or not it carries a membership, so the incoming-edge condition holds for records outside $\mathcal{A}$; this is exactly the RDF-star reading of a quoted, unasserted triple. Time statements differ from one another and from record vertices by subject or reserved predicate, so isomorphism can only arise between records with equal content. The argument of Theorem 8 gives the rest; decoding inverts each step, and $\mathcal{A}$ is recovered from the record memberships. □

With $S = \mathsf{K}(D)$ this is a lossless roundtrip of the exportable bitemporal history, and of the whole history when D is acyclic (Proposition 6). Two choices of $\mathcal{A}$ are natural. Taking $\mathcal{A} = S$ presents every occurrence, including retracted ones, as asserted; RDF-star consumers that ignore τ_d then see retracted facts as true. Taking $\mathcal{A} = S \cap D@t$ asserts the snapshot and quotes history; consumers that ignore time see exactly the snapshot, and the history remains attached as statements about quoted triples. This matches the RDF 1.2 separation of a proposition from its assertion [8], although our image is the cited RDF-star one, not RDF 1.2 reifiers. Each record costs up to nine additional statement vertices; sharing one time statement per distinct clock value would reduce that, at the price of a less direct decoder.

Graph memberships of the source. A tiramemsu membership $(m, e, \texttt{sys:inGraph}, g)$ is an ordinary record and is encoded as an ordinary triple, so its annotations and statement-valued graph names survive structurally. Its native named-graph meaning is not mapped to the target *in*; recovering the source dataset view is a separate interpretation step after decoding.

8 Validation

The accompanying `validate.py` uses only the Python standard library. It compares three kernel implementations, the peel, the component-based version of revision 2 and literal Kleene iteration of F_S, with an oracle that enumerates all subsets and tests closure and acyclicity by transitive closure. For each kernel it checks that the peel order is topological, builds E_f, checks the image conditions and identity preservation, decodes, and compares with temporal erasure, the RDF-star duplicate criterion and idempotence.
The run checked **217 625** labeled candidate stores of up to three records, **72 567** four-record reference structures up to relabeling, **20 000** random stores of four to seven records, and **1 500** random protocol histories of 8 transactions (66 000 snapshot checks, 853 states containing replay cycles); 3 000 reference-closure minimality checks, 10 focused store cases, 5 rejected image mutations and the same-transaction boundary also passed; the two engine traces were recomputed from rows dumped by tm-core 0.4.0, with no disagreements.

The labeled stores use up to three records over one node, one literal, two predicates, all present eids and one missing eid. The four-record structures enumerate every subject and object reference pattern, with one missing eid, up to relabeling; they include two-cycles with two-deep dependent chains, which three records cannot exhibit. The protocol histories apply random inserts obeying C3, cascading retractions and supersessions with random patches, including patches that create replay cycles. After each transaction they check Proposition 6 (closure, cycles inside one replay batch, the form of $\mathsf{K}(D)$), Proposition 7 (bit equals kernel; closed snapshots are filtered by the bit), Proposition 5 for histories without supersession, Proposition 4 for every snapshot (the closure exists, adds only records retracted by then, and the kernel inclusions hold), and Theorem 9 with $\mathcal{A} = \mathsf{K}(D) \cap D@t$. A separate check compares $\mathsf{C}_D(S)$ with the intersection of all closed supersets.

Engine traces. The test `crates/tm-core/tests/export_boundary.rs` runs both corrections of the running example on the engine, on its native SQLite host and on a minimal host, and asserts the kernels stated in Examples 1 and 2 using a test-local peel. With `TM_EXPORT_TRACE` set it dumps the raw records; the validator recomputes both kernels from the dump with its own implementations and checks the temporal roundtrip on them. The engine was the repository working tree with version string 0.4.0, built on commit `5a55efc`.

9 Related work

Statement identity. Reification, singleton properties and RDF-star describe statements in different ways [7, 9, 15]; RDF 1.2 reifiers separate a proposition from its assertion [8]. Named graphs can carry one statement each and attach provenance to the graph name [2]. Wikidata statements are identified objects with qualifiers and references, and Hernández et al. compare their encodings in RDF, including reification, singleton properties and per-statement named graphs [9, 22]. Multilayer graphs and MillenniumDB make identified edges available as endpoints [1, 23]. The occurrence record belongs to this family; what we add is the export boundary for a temporal variant of it.

Temporal stores. Datomic stores datoms (e, a, v, tx, op) in an accumulate-only log and answers as-of and history queries [3]; a datom has a transaction but no identity of its own that other datoms can reference, so the closure question does not arise in that form. XTDB is bitemporal at the document level [11], and SQL:2011 adds system-versioned and application-time period tables [13]; in both, references are foreign keys between rows rather than statements about statements. Gutierrez, Hurtado and Vaisman label RDF triples with valid time and give a semantics and

query language for temporal RDF [6]; our records carry two clocks per occurrence and allow occurrences as endpoints, which is what makes closure and cycles relevant.

Statement Graphs and OneGraph. OneGraph motivates interoperability and updates through different surface views [14]; Statement Graphs provide the image definitions used here [4, 5]; the Neptune demo applies OneGraph concepts in a product [21]. Our encoding is one prescribed transformation into one branch of that framework.

Companions. The temporal companion distinguishes structural support from historical lineage and studies correction [17]. Its closure calculus motivates the kernel, which additionally excludes cycles and their dependents to meet the DAG condition. The metagraph companion concerns different exact images [18].

10 Limitations

The proofs are by hand and the validator is a finite model check, not a mechanized proof. Engine evidence is limited to the two traced examples and to the engine's documentation and tests; the kernel is implemented in a test and in the validator, not as an engine API. Valid-time selection is outside the closure results. The encodings target the RDF-star branch of the cited OneGraph image: we claim neither an LPG image theorem, which would need role-specific term conversion and label statements, nor full RDF 1.2 interchange, nor any equivalence of SPARQL or Cypher query answers between the store and its export. The source dataset view and the named-graph meaning of source memberships are not transferred. The inherited storage measurements in Appendix B are exploratory and were not rerun.

11 Conclusion

A temporal statement store is a Statement Graph exactly on its closed, acyclic record sets. Every selection has a greatest such subset, the least fixpoint of a one-line operator, computable by a topological peel and maintainable as one bit per record. Exporting the full history rather than a snapshot removes the lineage failure, so the only records an append-only store cannot export are replay cycles and their dependents, and a store that rejects cyclic corrections loses nothing. With time encoded as reserved statements, the exportable history roundtrips through the OneGraph image without loss. An exporter built on these results should state which of kernel and reference closure it applies, return each omitted record with its reason, and ship the dictionary its decoder requires.

Availability. The source, validator, engine trace, generated counts and PDF are in `paper/statement-as-one-record/` of `https://github.com/Volland/tiramemsu`; the engine test is `crates/tm-core/tests/export_boundary.rs`. Tiramemsu is licensed under MIT OR Apache-2.0.

A Design commitments of tiramemsu

C3–C6 are stated in Section 6. The others describe the implementation and do not enter the theorems.

Commitment	Enforcement and boundary
C1 Exact arity	Tuple fields fix one target per role; indexes and dictionary entries are additional physical structures.
C2 IRI predicates	Abstract term type and engine validation; an integer SQL column alone does not establish IRI typing.
C7 Two clocks	Lifetime and valid interval are columns; valid-time containment between referring records is not required.
C8 Occurrences	Parallel content can have distinct eids, episodes and annotations; a triple view collapses them.
C9 Addressability	Any occurrence may be referenced, including property and membership records outside a direct LPG view.
C10 Predicate schema	Optional cardinality, uniqueness and type policies constrain writes; they are not consequences of row shape.

B Inherited layout measurements

Table 2 reproduces an earlier article [16] and was not rerun. The script `bench/eid-vs-reifier/bench.py` compares a SQLite layout with an eid rowid and three content indexes (A) against a clustered (s, p, o) key with two permutation indexes and a triple-term table (B), on 700 000 synthetic triples, 50 000 annotations and 500 annotations of annotations. Neither layout has time or graph columns. Both use `synchronous=OFF`, A is timed before B, the write workloads use different targets, most query results are not compared, and no hardware or variance is recorded. The numbers do not isolate the cost of an identifier and support no claim about durable writes, temporal queries or Neptune.

Table 2: Exploratory timings reproduced from [16].

Measure	A: eid layout	B: reifier layout
File size	53.6 MB	35.5 MB
Point lookup	2.7 μs	2.8 μs
A node's edges and confidence	6.5 μs	8.7 μs
All edges with confidence ≥ 95	2.1 ms	5.2 ms
Write an edge and confidence	35 μs	47 μs

References

[1] Renzo Angles, Aidan Hogan, Ora Lassila, Carlos Rojas, Daniel Schwabe, Pedro Szekely, and Domagoj Vrgoč. Multilayer graphs: A unified data model for graph databases. In *Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–6, 2022. doi: 10.1145/3534540.3534696.

[2] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs, provenance and trust. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, pages 613–622, 2005. doi: 10.1145/1060745.1060835.

[3] Cognitect / Nubank. Datomic documentation: Data model and transaction model. Software documentation. URL https://docs.datomic.com/. Accessed 11 October 2026.

[4] Ewout Gelling, George Fletcher, and Michael Schmidt. Bridging graph data models: RDF, RDF-star, and property graphs as directed acyclic graphs [extended abstract], 2023. URL https://arxiv.org/abs/2304.13097.

[5] Ewout Gelling, George Fletcher, and Michael Schmidt. Statement graphs: Unifying the graph data model landscape. In *Database Systems for Advanced Applications (DASFAA 2024)*, Lecture Notes in Computer Science, pages 364–376. Springer, 2024. doi: 10.1007/978-981-97-5575-2_27.

[6] Claudio Gutierrez, Carlos A. Hurtado, and Alejandro Vaisman. Introducing time into RDF. *IEEE Transactions on Knowledge and Data Engineering*, 19(2):207–218, 2007. doi: 10.1109/TKDE.2007.34.

[7] Olaf Hartig. Foundations of RDF⋆ and SPARQL⋆ (an alternative approach to statement-level metadata in RDF). In Juan Reutter and Divesh Srivastava, editors, *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*, volume 1912 of *CEUR Workshop Proceedings*, 2017. URL https://ceur-ws.org/Vol-1912/paper12.pdf.

[8] Olaf Hartig, Pierre-Antoine Champin, Andy Seaborne, and Gregg Kellogg. RDF 1.2 concepts and abstract data model. W3C Candidate Recommendation Snapshot, April 2026. URL https://www.w3.org/TR/2026/CR-rdf12-concepts-20260407/. 7 April 2026; cited names are editors; Gregg Kellogg served until 6 September 2025.

[9] Daniel Hernández, Aidan Hogan, and Markus Krötzsch. Reifying RDF: What works well with Wikidata? In *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems (SSWS)*, volume 1457 of *CEUR Workshop Proceedings*, pages 32–47, 2015. URL https://ceur-ws.org/Vol-1457/SSWS2015_paper3.pdf.

[10] Richard Jones and Rafael Lins. *Garbage Collection: Algorithms for Automatic Dynamic Memory Management*. Wiley, 1996.

[11] JUXT. XTDB documentation: Bitemporality. Software documentation. URL https://docs.xtdb.com/. Accessed 11 October 2026.

[12] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5 (11):558–562, 1962. doi: 10.1145/368996.369025.

[13] Krishna Kulkarni and Jan-Eike Michels. Temporal features in SQL:2011. *SIGMOD Record*, 41(3):34–43, 2012. doi: 10.1145/2380776.2380786.

[14] Ora Lassila, Michael Schmidt, Olaf Hartig, Brad Bebee, Dave Bechberger, Willem Broekema, Ankesh Khandelwal, Kelvin Lawrence, Carlos Manuel Lopez Enriquez, Ronak Sharda, and Bryan Thompson. The OneGraph vision: Challenges of breaking the graph model lock-in. *Semantic Web*, 14(1):125–134, 2023. doi: 10.3233/SW-223273.

[15] Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. Don't like RDF reification? Making statements about statements using singleton property. In *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, pages 759–770, 2014. doi: 10.1145/2566486.2567973.

[16] Volodymyr Pavlyshyn. How tiramemsu differs from oxilite, 2026. URL https://tiramemsu.com/articles/tiramemsu-vs-oxilite.html. Site article, 30 September 2026; source of the reproduced exploratory measurements.

[17] Volodymyr Pavlyshyn. Layered bitemporal graphs: Statement identity, two clocks, and metagraphs as tags. Preprint, revision 4, 3 October 2026, 2026. URL https://doi.org/10.5281/zenodo.23139228.

[18] Volodymyr Pavlyshyn. Metagraph encodings over identified triples: Exact images, temporal integrity, and structural depth. Preprint, revision 2, 5 October 2026, 2026. URL https://github.com/Volland/tiramemsu/tree/main/paper/metagraphs-from-identified-triples.

[19] Volodymyr Pavlyshyn. Statement graphs and tiramemsu: one idea, one row deep, 2026. URL https://tiramemsu.com/articles/statement-graphs.html. Site article, 6 October 2026.

[20] Volodymyr Pavlyshyn. tiramemsu: an embedded bitemporal graph database with statement identity. Software, version 0.4.0, MIT OR Apache-2.0, 2026. URL https://tiramemsu.com.

[21] Michael Schmidt, Brad Bebee, Willem Broekema, Mohamed Elzarei, Carlos Manuel Lopez Enriquez, Marcin Neyman, Florian Schmedding, Andreas Steigmiller, Bryan Thompson, Geo Varkey, Gregory Todd Williams, and Amanda Xiang. openCypher over RDF: Connecting two worlds. In *Proceedings of the ISWC 2024 Posters, Demos and Industry Tracks: From Novel Ideas to Industrial Practice*, volume 3828 of *CEUR Workshop Proceedings*, 2024. URL https://ceur-ws.org/Vol-3828/paper29.pdf.

[22] Denny Vrandečić and Markus Krötzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014. doi: 10.1145/2629489.

[23] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. MillenniumDB: An open-source graph database system. *Data Intelligence*, 5(3):560–610, 2023. doi: 10.1162/dint_a_00229. Preprint: arXiv:2111.01540.